Lab: Campus Library System

Inheritance, Abstract Classes, Mixins, Enums, Extensions, Operators & static/const

Duration	90 minutes
Format	Individual work, pure Dart. Every student has personalised numbers.
Slides	Session 1, slides 1 to 16 (Inheritance through Common Pitfalls)
Tools	Dart SDK (`dart run --enable-asserts`), Git, your existing course repository on GitHub
Submission	`git push` to your course repo after every step (10 pushes), plus a final push with answers
Total marks	100

Learning Objectives

By the end of this lab you will be able to:

- Build a class hierarchy with `extends`, `super` parameters and `@override`
- Declare an abstract class and force subclasses to implement its methods
- Use `implements` for a contract and `with` for a mixin, and say when each is right
- Model a fixed set of choices with an enhanced enum
- Add methods to existing types with extensions
- Overload `+` and `>`, and write `==` together with `hashCode`
- Use `static` members and `const` constructors, and show that const objects are shared

Your Personal Version of This Lab

Every student gets **different numbers**, so every student's printed answers are different. The starter code reads the **last two digits of your roll number** and turns them into a seed:

- seed is the two-digit number (00 to 99). For roll number 21F-0347 the seed is 47.
- t is the tens digit (4) and u is the units digit (7).

Late fees, days, caps and shelf sizes are all calculated from t and u. Output that is correct for another roll number will **not** match yours, and that is intended.

Rules: do not edit the seeded settings in the starter file; never use `t` or `u` as names of your own variables; use whole-number (int) arithmetic only, and `~/` for division. Your instructor checks your code by running it and comparing it with an answer key generated for your own roll number.

Time Plan and Checkpoints

Each step ends with a **checkpoint**: you commit and push. The "finish by" column is the minute (from the start of the lab) by which your push should be on GitHub. GitHub records when each push arrived.

Checkpoint	Topic	Time	Finish by
Setup	Create the folder and file, first push	6 min	min 6
Step 1	Classes and `extends`	5 min	min 11
Step 2	`super` and `@override`	7 min	min 18
Step 3	Abstract classes	8 min	min 26
Step 4	`implements` (an interface)	7 min	min 33
Step 5	Mixins with `with`	7 min	min 40
Step 6	Enhanced enums	6 min	min 46
Step 7	Extension methods	6 min	min 52
Step 8	Operator overloading, `==` and `hashCode`	8 min	min 60
Step 9	`static` and `const`	8 min	min 68
Step 10	Capstone: the whole library	11 min	min 79
Wrap-up	Format, analyse, answers, final push	11 min	min 90
Total		**90 min**	

How to submit: your GitHub repository

You already have a repository for this course. Do **not** create a new one. Put everything for this lab in a folder named `oop_lab` at the top level of that repository:

```
your-course-repo/
  oop_lab/
    library_lab.dart      <- your program
    ANSWERS.md            <- reflection answers (at the end)
    output.txt            <- your program's printed output (at the end)
```

At the end of **every step**, run these commands from the top of your repository (replace N):

```
git add oop_lab
git commit -m "OOP lab: step N"
git push
```

Integrity rules: never force-push, rebase or amend a commit that you already pushed. Your commit history is your proof that you did the work in order and in time. Commit messages must follow the format above.

Setup (6 min)

- Open a terminal in your course repository and run `git pull` so you start up to date.
- Create the folder `oop_lab` and the file `oop_lab/library_lab.dart`. Paste the starter code below (your instructor also shares it as `library_lab_starter.dart`).
- Replace the roll number text with **your** roll number, which must end in two digits.
- Run `dart run --enable-asserts oop_lab/library_lab.dart`. You should see your seed line and ten step headings.
- Commit with the message `OOP lab: setup` and push.

```
// library_lab.dart  -  Campus Library System
// Name: _____________________   Roll no: ____________

const String rollNo = 'YOUR-ROLL-NO-HERE';   // e.g. '21F-0347'

// ===== Seeded settings (generated from YOUR roll number). Do not edit. =====
final int seed = int.parse(rollNo.substring(rollNo.length - 2));
final int t = seed ~/ 10;   // tens digit
final int u = seed % 10;    // units digit

const List<String> titles = [
  'Dart Basics', 'Flutter In Action', 'Data Structures', 'Operating Systems',
  'Mobile Design', 'Clean Code', 'Algorithms Unlocked', 'Computer Networks',
  'Database Systems', 'Software Testing',
];

int yearOf(int i) => 1995 + 2 * i + t;      // publication year of titles[i]
final int feePerDay = 5 + t;                // late fee per day, in rupees
final int daysLate = 3 + (t + u) % 6;       // normal number of days late
final int daysBig = 10 + u;                 // a long overdue period
final int feeCap = 150 + 10 * t;            // no single fee may exceed this
final int renewLimit = 1 + t % 3;           // renewals allowed per item
final int renewAttempts = 2 + u;            // renewal attempts made in Step 5
final int shelfSize = 3 + u % 4;            // items on the shelf in Steps 9-10
// ============================================================================

void main() {
  print('Seed: $seed (t=$t, u=$u)');
  step1();
  step2();
  step3();
  step4();
  step5();
  step6();
  step7();
  step8();
  step9();
  step10();
}

void step1() { print('--- Step 1 ---'); }
void step2() { print('--- Step 2 ---'); }
void step3() { print('--- Step 3 ---'); }
void step4() { print('--- Step 4 ---'); }
void step5() { print('--- Step 5 ---'); }
void step6() { print('--- Step 6 ---'); }
void step7() { print('--- Step 7 ---'); }
void step8() { print('--- Step 8 ---'); }
void step9() { print('--- Step 9 ---'); }
void step10() { print('--- Step 10 ---'); }
```

If you get a FormatException straight away, you have not replaced the placeholder roll number, or it does not end in two digits. Put your classes above `main()`, and your step code inside the matching `stepN()` function. Keep every `print` exactly as shown: the checker looks for lines that start with `Step N:`.

Step 1: Classes and extends (5 min)

Topic: Inheritance. A subclass takes everything from its parent and adds its own parts.

Task 1.1

Write a class `LibraryItem` with two fields, `final String title;` and `final int year;`, and a constructor `LibraryItem(this.title, this.year).`

Task 1.2

Write `class Book extends LibraryItem` with one extra field `final int pages;`. Use a **super parameter** so the parent gets the first two values: `Book(super.title, super.year, this.pages);`

Task 1.3

Write a top-level function `Book mainBook()` that returns `Book(titles[u], yearOf(u), 100 + 3 * seed)`. You will reuse it in every later step. In `step1()` call it, then store the result in a variable declared as `LibraryItem item = b;` and print:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 1: <title> (<year>), <pages> pages
Step 1: stored as LibraryItem: <item.title>
Step 1: runtimeType = <item.runtimeType>
```

Self-check: the third line says Book even though the variable type is LibraryItem.

Think (answer in a code comment): Why is `item.pages` a compile error even though the object really has pages?

PUSH. Checkpoint 1: commit message `OOP lab: step 1`, then `git push`.

Step 2: super and @override (7 min)

Topic: Customising behaviour. `@override` replaces a parent method; `super.method()` runs the parent's version first.

Task 2.1

Add to `LibraryItem` the method `String describe() => '$title ($year)';`

Task 2.2

In `Book`, override it so it **reuses** the parent's version: `@override String describe() => '${super.describe()}, $pages pages';`

Task 2.3

Write `class Magazine extends LibraryItem` with `final int issue;` and its own `describe()` that ends with `, issue <issue>`. Add the top-level function `Magazine mainMagazine()` that uses index `i = (u + 1) % 10` and returns `Magazine(titles[i], yearOf(i), 1 + (t + u) % 12)`. Print:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 2: <mainBook().describe()>
Step 2: <mainMagazine().describe()>
```

Self-check: the first line is your Step 1 line with the title and year in front of the pages.

Think (answer in a code comment): What would happen to the output if you forgot `@override` and mistyped the method name as `describe()`?

PUSH. Checkpoint 2: commit message `OOP lab: step 2`, then `git push`.

Step 3: Abstract Classes (8 min)

Topic: Abstract classes. A promise with no body: every concrete subclass must keep it.

Task 3.1

Make `LibraryItem` abstract and add a method with **no body**: `int lateFee(int days);`

Task 3.2

Implement `lateFee` with `@override` in all three subclasses. Add a new subclass `Dvd extends LibraryItem` with `final int minutes;` and its own `describe()` ending with `, <minutes> min`.

- Book: `days * feePerDay`
- Magazine: `days * (feePerDay ~/ 2)`
- Dvd: `days * feePerDay * 2`

Write `Dvd mainDvd()` that uses index `i = (u + 2) % 10` and returns `Dvd(titles[i], yearOf(i), 90 + seed % 40)`.

Task 3.3

In `step3()` print the late fee of each item for `daysLate` days:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 3: Book late fee for <daysLate> days = <fee>
Step 3: Magazine late fee for <daysLate> days = <fee>
Step 3: Dvd late fee for <daysLate> days = <fee>
```

Self-check: the Dvd fee is exactly double the Book fee.

Think (answer in a code comment): Temporarily try `LibraryItem('x', 2000)`. Read the error. Why can you not create an abstract class directly?

PUSH. Checkpoint 3: commit message `OOP lab: step 3`, then `git push`.

Step 4: implements: A Contract (7 min)

Topic: extends vs implements. `implements` takes only the shape of a class; you write every method yourself.

Task 4.1

Write an abstract class `Borrowable` with three members and no bodies: `bool get isAvailable;`, `void borrow();`,`void giveBack();`

Task 4.2

Change `Book` so it also promises the contract: `class Book extends LibraryItem implements Borrowable`. Add a private field `bool _out = false;` and implement the three members with `@override`: `isAvailable` is `!_out`, `borrow()` sets `_out` to true, `giveBack()` sets it to false. Magazines and Dvds stay as they are.

Task 4.3

In `step4()` create `book = mainBook()`, print its availability, borrow it, print again, return it, print again. Then put `book`, `mainMagazine()` and `mainDvd()` in a `List<LibraryItem>` and print one line per item using `item is Borrowable`:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 4: available? <book.isAvailable>
Step 4: after borrow? <book.isAvailable>
Step 4: after return? <book.isAvailable>
Step 4: <item.runtimeType> borrowable? <item is Borrowable>      (three lines)
```

Self-check: exactly one of the three items is borrowable.

Think (answer in a code comment): You wrote all three method bodies yourself. What would `extends Borrowable` have given you for free, and why is that not what you wanted?

PUSH. Checkpoint 4: commit message `OOP lab: step 4`, then `git push`.

Step 5: Mixins with with (7 min)

Topic: Mixins. Share ready-made code sideways, between classes that are not related by inheritance.

Task 5.1

Write a mixin that counts renewals:

```
mixin Renewable {
  int renewals = 0;
  bool renew(int limit) {
    // TODO: if renewals is already >= limit, return false.
    // Otherwise add one to renewals and return true.
  }
}
```

Task 5.2

Add the mixin to `Book` and to `Dvd` (but not to `Magazine`). The order is: `extends` first, then `with`, then `implements`. For example `class Dvd extends LibraryItem with Renewable`.

Task 5.3

In `step5()` take a fresh `book = mainBook()` and call `book.renew(renewLimit)` exactly `renewAttempts` times, counting how many calls returned true. Then try once more and test a Dvd:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 5: renewed <ok> of <renewAttempts> attempts (limit <renewLimit>)
Step 5: renew once more? <book.renew(renewLimit)>
Step 5: dvd can renew too? <mainDvd().renew(1)>
```

Self-check: ok never exceeds renewLimit. For some roll numbers the “one more” call is still true, because you made fewer attempts than the limit.

Think (answer in a code comment): Book and Dvd need the same `renew` code but Magazine must not have it. Why is a mixin a better fit than putting `renew` in `LibraryItem`?

PUSH. Checkpoint 5: commit message `OOP lab: step 5`, then `git push`.

Step 6: Enhanced Enums (6 min)

Topic: Enums. A fixed set of named choices that can carry data and methods.

Task 6.1

Write this enum. Fill in the method:

```
enum MemberType {
  student(3, 50),
  faculty(10, 20),
  guest(1, 0);

  final int maxLoans;
  final int discountPercent;
  const MemberType(this.maxLoans, this.discountPercent);

  // TODO: int applyDiscount(int fee) -> fee minus discountPercent % of fee
  //       (use ~/ for the division)
}
```

Task 6.2

In `step6()` pick your member type with `MemberType.values[seed % 3]`. Use the book fee for `daysLate` days (from Step 3) and print:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 6: member type = <type.name> (index <type.index>)
Step 6: max loans = <type.maxLoans>
Step 6: book fee <fee> -> <type.applyDiscount(fee)> after discount
```

Self-check: a guest pays the full fee; a student pays half (rounded down).

Think (answer in a code comment): Why is `MemberType.student` safer than the string 'student'?

PUSH. Checkpoint 6: commit message `OOP lab: step 6`, then `git push`.

Step 7: Extension Methods (6 min)

Topic: Extensions. Add methods to types you did not write.

Task 7.1

Write three extensions:

```
extension MoneyText on int {
  String get rs => ...;          // 72.rs  ->  'Rs 72'
}
extension Initials on String {
  String get initials => ...;    // 'Computer Networks'.initials -> 'CN'
}
extension ShelfMath on List<LibraryItem> {
  int totalLateFee(int days) => ...;   // sum of lateFee(days) over the list
}
```

Hints: `split(' ')` splits a String into words, `map((w) => w[0])` takes the first letter of each, `join()` glues them. Use `fold(0, (sum, i) => ...)` for the total.

Task 7.2

In `step7()` use the discounted fee from Step 6, the title `titles[u]`, and a list `[mainBook(), mainMagazine(), mainDvd()]`:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 7: <discounted fee>.rs
Step 7: initials of "<titles[u]>" = <initials>
Step 7: shelf fee for <daysLate> days = <totalLateFee(daysLate).rs>
```

Self-check: the shelf fee equals the sum of your three Step 3 fees.

Think (answer in a code comment): You could have written plain functions `rs(int x)`. What do you gain by writing `72.rs` instead?

PUSH. Checkpoint 7: commit message `OOP lab: step 7`, then `git push`.

Step 8: Operators, == and hashCode (8 min)

Topic: Operator overloading. Teach `+` and `>` to your own class, and make equal values equal.

Task 8.1

Write a class `Fine` with `final int amount;` and a **const** constructor `const Fine(this.amount);`. Add:

- `Fine operator +(Fine o)` returning a new Fine with the sum
- `bool operator >(Fine o)` comparing the amounts
- `@override bool operator ==(Object other)` that is true when `other` is a Fine with the same amount
- `@override int get hashCode` that returns `amount.hashCode`
- `@override String toString() => 'Fine($amount)';`

Task 8.2

In `step8()` let `f1 = Fine(book fee for daysLate days)` and `f2 = Fine(magazine fee for daysLate days)`. Print:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 8: <f1> + <f2> = <f1 + f2>
Step 8: first bigger? <f1 > f2>
Step 8: equal copies? <f1 == Fine(f1.amount)>
Step 8: set size = <{f1, Fine(f1.amount), f2}.length>
```

Self-check: the set holds three values but only two are different, so its size is 2.

Think (answer in a code comment): Delete your `hashCode` and run again. What happens to the set size, and why does Dart expect both `==` and `hashCode` together?

PUSH. Checkpoint 8: commit message `OOP lab: step 8`, then `git push`.

Step 9: static and const (8 min)

Topic: Static and immutable. One value shared by the class; constant objects built once.

Task 9.1

Add two **static** members to `LibraryItem`: a counter `static int created = 0;` that the constructor increases by one (put `created++;` in a constructor body), and a helper `static int capFee(int fee)` that returns `feeCap` when `fee` is larger than `feeCap`, otherwise `fee`.

Task 9.2

Write `List<LibraryItem> buildShelf()` that creates `shelfSize` items. For k from 0 up to shelfSize - 1, let `i = (u + k) % 10` and create:

- k % 3 == 0: `Book(titles[i], yearOf(i), 100 + 3 * seed)`
- k % 3 == 1: `Magazine(titles[i], yearOf(i), 1 + (t + k) % 12)`
- otherwise: `Dvd(titles[i], yearOf(i), 90 + seed % 40)`

Task 9.3

In `step9()` first set `LibraryItem.created = 0;`, then call `buildShelf()` **twice** (keep the second result as `shelf`) and print `created` straight away. Then print the capped fee of `shelf.first.lateFee(daysBig)`. The last two lines compare `identical(const Fine(10), const Fine(10))` with `identical(Fine(10), Fine(10))` (no `const` on the second):

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 9: items created = <LibraryItem.created>
Step 9: capped fee = <capFee(shelf.first.lateFee(daysBig))> (cap <feeCap>)
Step 9: const copies identical? <true or false>
Step 9: new copies identical? <true or false>
```

Self-check: items created is exactly twice your shelfSize.

Think (answer in a code comment): Why can `created` be read as `LibraryItem.created` without making any object first?

PUSH. Checkpoint 9: commit message `OOP lab: step 9`, then `git push`.

Step 10: Capstone: The Whole Library (11 min)

Topics: everything so far, working together. Inheritance, abstract methods, mixin, interface, enum, operators and static in one small system.

Task 10.1

Write a class `Member` with `final String name;` and `final MemberType type;` and one method that gives the fine for one item:

```
Fine feeFor(LibraryItem item, int days) {
  // 1. fee = item.lateFee(days)
  // 2. cap it with LibraryItem.capFee
  // 3. apply the member's discount: type.applyDiscount
  // 4. return it as a Fine
}
```

Task 10.2

In `step10()` call `buildShelf()`, create `Member('S$seed', MemberType.values[seed % 3])`, and add up the fine for every item over `daysLate` days. Start from `Fine(0)` and use `fold` with the `+`

you wrote in Step 8. Count how many items are `Borrowable` and how many are `Renewable` with `is`. Print:

PRINT IN THIS FORMAT (replace <...> with your own values)

```
Step 10: <n> items, <b> borrowable, <r> renewable
Step 10: <member.name> (<member.type.name>) owes <total>
Step 10: over the cap? <total > Fine(feeCap)>
```

Self-check: one polymorphic loop over a `List<LibraryItem>` handles Books, Magazines and Dvds with no `if` on the type.

PUSH. Checkpoint 10: commit message `OOP lab: step 10`, then `git push`.

Wrap-up (11 min)

- Run `dart format oop_lab` and `dart analyze oop_lab`. Fix every warning.
- Run the whole program once and save the output: `dart run --enable-asserts oop_lab/library_lab.dart > oop_lab/output.txt`.
- Create `oop_lab/ANSWERS.md` with your name, roll number and the answers to these four questions (one or two sentences each):
  - **Q1.** What can a subclass using `extends` use for free that a class using `implements` has to write itself?
  - **Q2.** `Book` and `Dvd` both use `Renewable`. When would you pick a mixin, and when an abstract parent class?
  - **Q3.** Why must you override `hashCode` whenever you override `==`?
  - **Q4.** An extension method on `String` and a subclass of `String` both add behaviour. Why does Dart let you write the first one but not the second?
- Final push, commit message `OOP lab: final`:

```
git add oop_lab
git commit -m "OOP lab: final"
git push
```

Common Pitfalls

- A subclass with a required parent constructor needs `super` parameters or `: super(...)`, or you get `implicit_super_initializer_missing_arguments`.
- Forgetting `@override` means a typo silently creates a new method.
- Order matters: `extends` first, then `with`, then `implements`.
- An abstract method has no body. A concrete method in an abstract class does.
- `~/` is integer division; `/` gives a double and will not fit an int.
- If you override `==` without `hashCode`, sets and maps behave wrongly.
- `identical(Fine(10), Fine(10))` is false; only `const` objects are shared.

Marking Scheme

Section	Marks	What is checked
Steps 1 to 10 (8 marks each)	80	Output matches the answer key for the student's own seed (4, automatic); code uses the required feature correctly (3); the step was pushed by its checkpoint (1)
Final file	10	`library_lab.dart` runs, is formatted, has no analyzer warnings; `output.txt` is included
Reflection	10	Answers to Q1 to Q4 and the Think comments
Total	**100**	